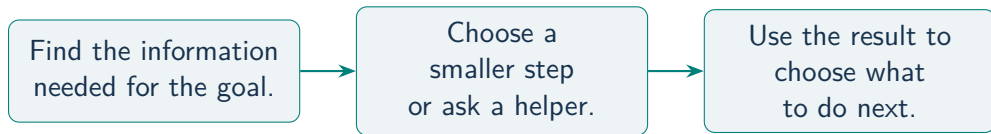


Can a model learn useful steps for an unfamiliar task?

A recursive language model (RLM) can use code and ask another model to handle a smaller task. A helper can, in turn, ask its own helper.



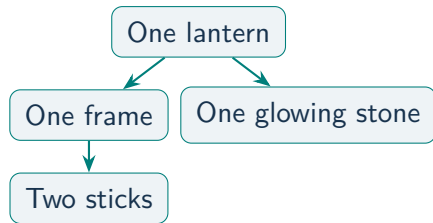
Our immediate question is simpler: **can we teach a small model to choose a useful next action from what it has actually seen?**

The strongest result so far concerns learning from examples. We have not yet shown that recursive helpers improve the whole task.

A crafting game makes the outcome easy to check.

In TextCraft, the model looks up recipes and makes items. The game checks whether the requested items really exist in its inventory.

Goal: make one lantern.



Arrows mean “needs.”

Simplified illustration, not a recorded task. Real tests use unfamiliar item names and longer chains.

Start with two sticks and one glowing stone.

1. Look up the lantern and frame recipes.
2. Make the frame from the sticks.
3. Combine the frame and stone into a lantern.

We test two ways of teaching the model.

Both change the model through training. The difference is the feedback it receives.

Worked examples (SFT)

We show a goal, observations, and a correct next action.

"The lantern needs a frame. Look up the frame recipe."

The model learns to choose that action.

Task rewards (RL)

The model tries the task. The game checks success.

"Did it actually make the lantern?"

Training favors attempts with better rewards.

SFT means supervised fine-tuning; RL means reinforcement learning. Our reward-training runs start from a model already trained on worked examples.

We changed the information shown before the action.

We reordered existing recipe lookups, then rebuilt what the learner sees before each action.

Original input

Goal: make a lantern.
No frame has been mentioned.



Next action to learn

Look up the frame recipe.

Repaired input

Goal: make a lantern.
Lookup: a lantern needs a frame.



The same action to learn

Look up the frame recipe.

We keep the same demonstrated actions and the same number of training updates. Illustration. The repair uses an existing solution. Input histories and their lengths change; computing work is not held exactly equal.

The repair helped after retraining and on more goals.

Successful attempts out of 32, with the same demonstrated actions and number of training updates:



There are **16 distinct goals across this chart**, with repeated attempts and changed recipes. The first two rows reuse the same goals.

These are small exploratory comparisons with Qwen3-4B. More training also helps the original examples. The result is better learning at the tested budgets, not a universal advantage.

Reward training showed little transfer so far.

Successes before → after one reward-based training update, out of 16 attempts:

Tool setting	Familiar goals	Different goals
Model writes ingredient list	9 → 13	4 → 5
Code fills ingredient list	14 → 16	5 → 5

Familiar goals were used in this reward-training update. The other target items were excluded, but component recipes may overlap.

Each column has eight goals and two attempts per goal. Compare the before-and-after change **within** a column; their difficulty differs.

Code uses recipes already observed; the model still chooses what and how much to make. This is one update per setting, not evidence that RL cannot generalize. More varied training and extra SFT are being tested.

Does a helper improve the whole task?

RLMs already support helpers. Here we test whether one helper earns its cost.

Work alone.

The main model looks up recipes, makes the frame, and assembles the lantern.

Use one helper.

A programmed rule assigns a helper: "Make one frame."

The helper reports success or what prevented completion.

The main model resumes with the updated inventory.

The original goal must succeed, not just the helper's subtask.

Planned test, not a result. Both share the same total allowance for model calls and generated text. The first rules for using a helper are programmed, not learned.

The strongest lead is better worked examples.

A possible focused paper

Repair what a learner sees before each demonstrated action, while keeping the taught actions fixed. Test when that makes learning more effective.

Next evidence we need: test the repair in another model family and on fresh goals and recipes. Compare broader reward training with more worked examples.

The RLM question remains: does delegating a useful part improve the whole task before we add deeper recursion?

For discussion: Would broader transfer or a clearer explanation of the teaching effect make the stronger first paper?

Prior work already studies what teachers and learners know. The candidate contribution is this controlled repair and its evidence, not the general idea. All results remain preliminary.